Architectures de Réseaux de Neurones Avancées

Segmentation sémantique

Segmentation sémantique

I. Introduction

II. Modèles de Deep Learning

III. Fonctions de coût

IV. Métriques

V. Post-traitement & tricks

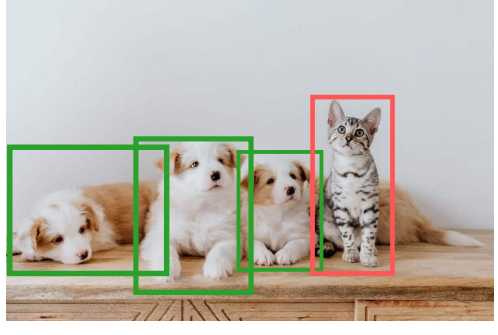
Qu’est-ce que la segmentation des images ?

Classification

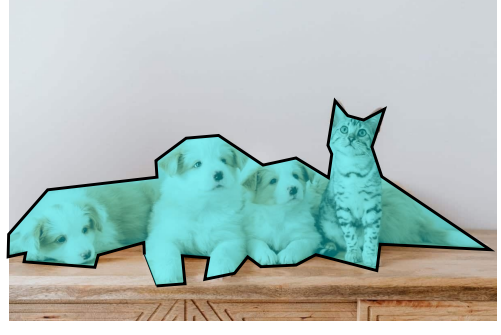


Cat, dog

Detection =
Localization + Classification

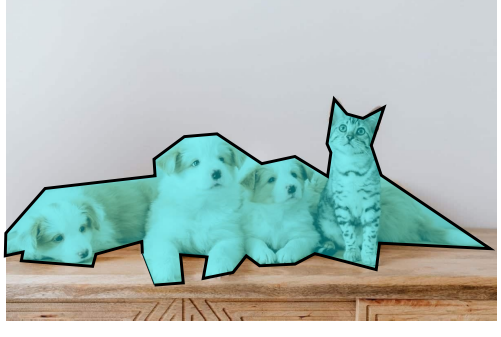


Segmentation



Qu’est-ce que la segmentation *sémantique* des images ?

Segmentation



Semantic Segmentation



Cat, dog

Instance (Semantic) Segmentation



Dog, dog, dog, cat

La tâche de la segmentation semantique

- Entrées → images (couleurs) de taille $H \times W \times 3$
- Sorties → masques de taille $H \times W \times K$ où K es le nombre de classe à reconnaître
- La dernière dimension contient les probabilités d'appartenance aux classes
- Masques binaires obtenus en prenant l'argmax sur la dernière dimension
- Entraînement supervisé:
    - On fournis une paire (image, masque binaire),
    - on entraîne le réseau de neurones à produire des probabilités d'appartenance aux classes qui correspondent à ces masques binaires
    - Il s'agit d'un problème de classification: chaque pixel à une classe associée

Bonus: Extension à la segmentation d’instance

- Aussi appelée segmentation panoptique
- Classiquement effectuée en 2 étapes:
    1. Détection des objets (classification + localisation)
    2. Segmentation de chacune des imagettes d'objet
- Il existe aussi des méthodes réalisant cette tâche en 1 fois
    - Repose généralement sur de multiples sorties spécialisées (cartes de saillances, prédiction de la forme des objets, classification de chaque pixel, ...)

Deep Learning models

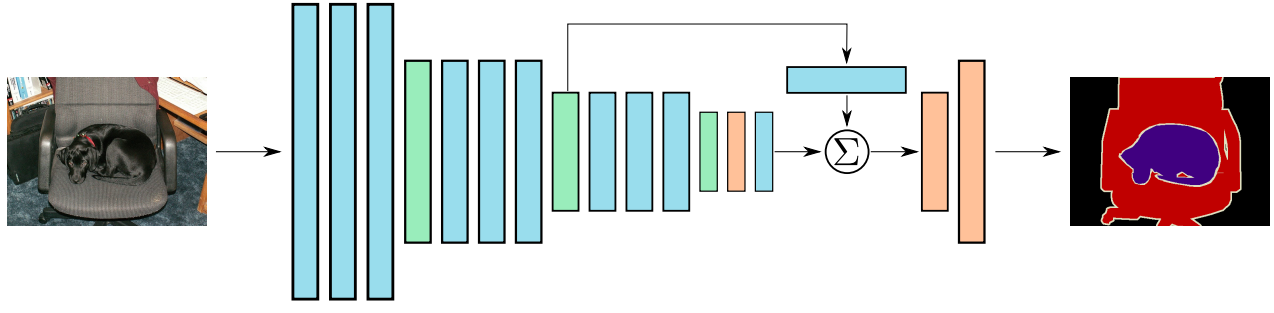
II.1. FCN (Fully Convolutional Network)

II.2. DeepLab

II.3. U-Net

Fully Convolutional Network (FCN)

- Pour une architecture donnée avec N changements de résolution, FCN-2^M est construit en fusionnant la sortie du réseau avec la sortie à l'échelle M
- Construction du réseau
    - Prendre la sortie à l'échelle N (image réduite d'un facteur 2^N)
    - Sur-échantillonner pour atteindre la même résolution que celle de l'échelle M (augmenter d'un facteur 2^{N-M})
    - Sommer les deux sorties et sur-échantillonner d'un facteur M pour obtenir la même taille que l'entrée



- Réseaux de convolutions généralement employés: VGG16 and GoogleNet ($N = 5$)

FCN limitations

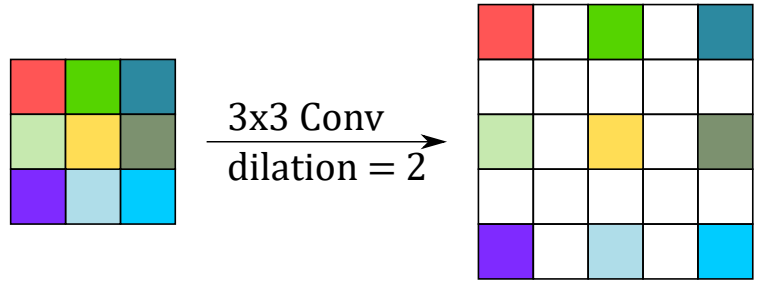
- FCN est complexe à correctement entraîner
    - M trop petit: Sur-échantillonnage à l'échelle N est trop grossier et génère la plupart des erreurs
    - M trop grand: Trop de détails perdus (**data processing inequality**)
- **Data processing inequality**: L'information contenue dans un signal ne peut pas être augmentée avec des traitements locaux
- **Solution**: Réseaux + profond avec moins de sous-échantillonnage + stratégies pour réduire le coût en calculs → DeepLab V1 (2015)

DeepLab V1

- Même architecture que FCN (base VGG16 ou GoogleNet) mais sans les deux derniers pooling
- Essaie de résoudre les problèmes de FCN
    - Feature maps plus large pour une profondeur de réseau donnée
    - Utilisation de plusieurs échelles pour casser l'inégalité de traitement des données
- Solutions proposée dans DeepLabV1:
    1. **Convolutions dilatées** (aka "atrous convolution" du français "à trous")
    2. **Conditional Random Fields** post-traitement **global** des cartes de probabilités
    3. **Multi-échelles**: Add more scales (4) than just 1 compared to FCN

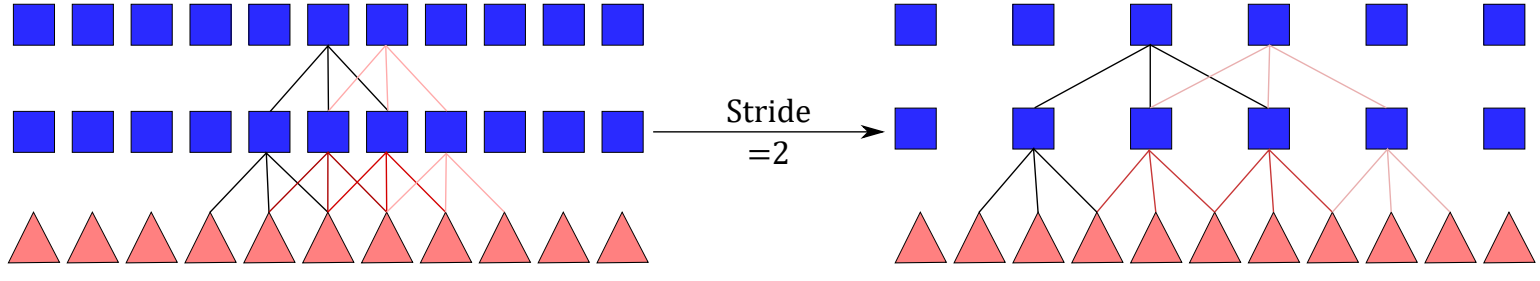
Convolution dilatée

- **Convolution creuse** (sparse convolution): càd avec des zéros
- Idée principale: peu de paramètres sur un noyau plus grand
- Implémentation: `torch.nn.Conv2d(3, 32, (3, 3), dilatation=(2, 2))`
- Exemple: convolution avec un noyau 3x3 étendu à un noyau 5x5 (9 paramètres au lieu de 25)



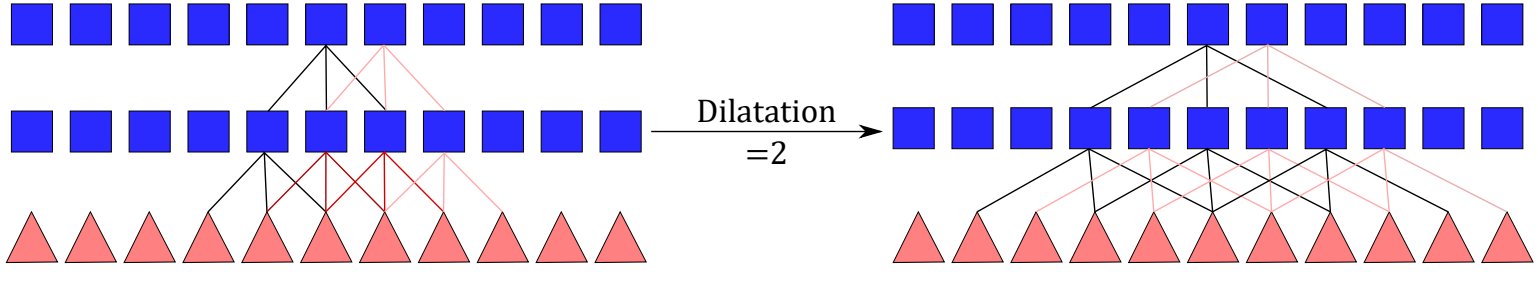
Dilatation VS stride

- **Stride et pooling réduisent la resolution spatiale**
- Convolution de taille 2r+1 avec stride de 2, suivie d'une convolution de taille 2r+1
    - Champs réceptif = 2 * 2r + 2r + 1
- Ex: Conv 3x3 (r=1) → champs réceptif de taille 7
    - Chevauchement important avec les convolutions successives



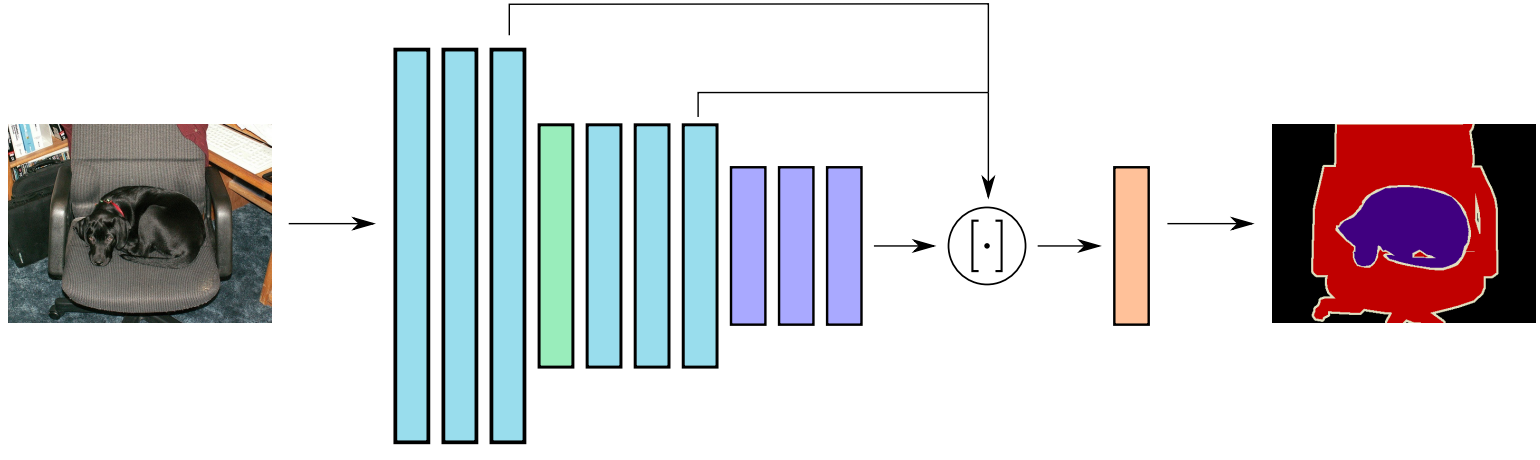
Dilatation VS stride

- **La dilatation ne réduit pas la résolution**
- 2 convolutions successives de taille 2r+1 dilatée d'un facteur 2
    - Champs réceptif = 2 * 2r + 2 * 2r + 1
    - Conv 3x3 (K=1) = 9
- Champ réceptif + grand et résolution conservée
- La convolution dilatée coûte 4 fois le coût en calcul et en mémoire d'une convolution classique



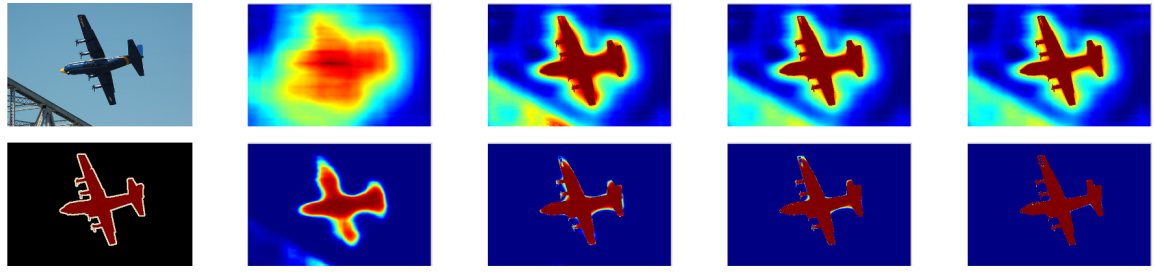
Connections multi-échelles

- Similaire à FCN → Sur-échantillonnage pour correspondre à la dimension spatiale
- Au lieu de le faire à 1 seule échelle → 4 échelles
- Somme remplacée par la concaténation → augmente la dimension mais améliore les capacités de généralisation



Post-traitement: Conditional Random Fields

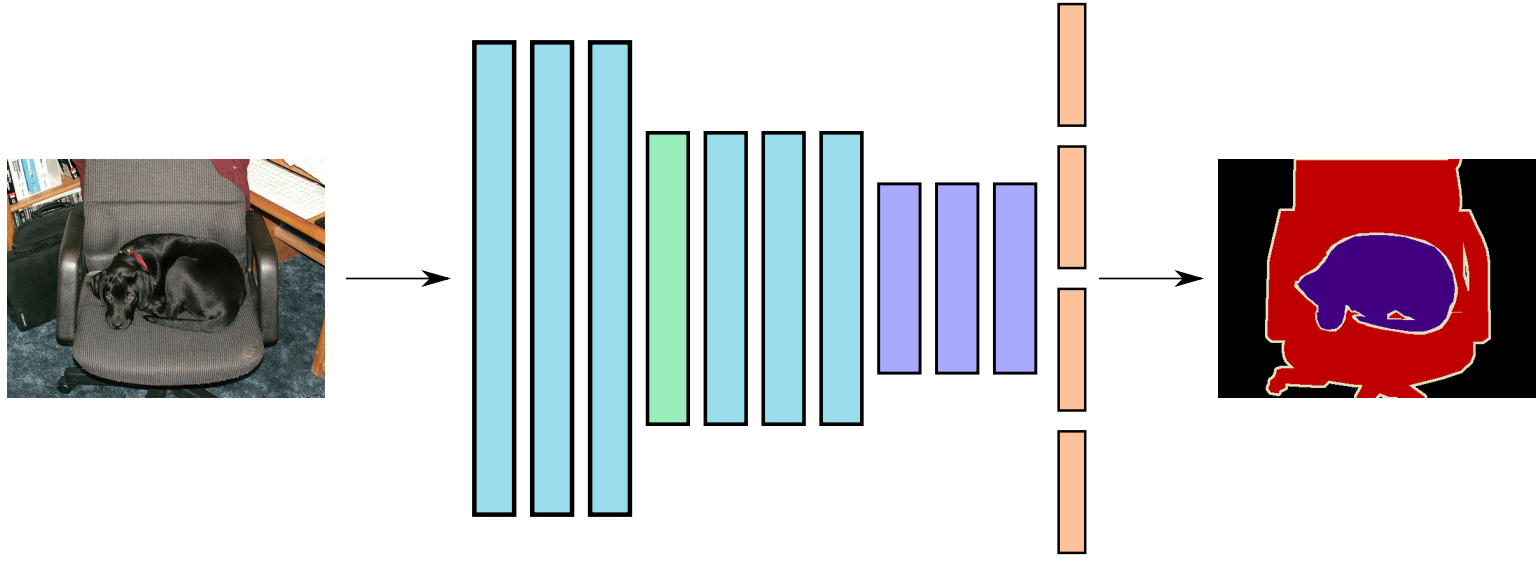
- Conditional Random Fields (CRFs): outil classique pour améliorer la qualité de la segmentation
- Repose sur un graphe de voisinage → pixels avec des prédictions similaires sont renforcés pour être semblables
- Permet de nettoyer les pixels dont les prédictions erronées serait facilement détectable (ex: groupe de pixels à l'interieur d'un objet) dû au fait que chaque pixel est prédit indépendamment des autres



Comparaison entre la sortie de réseau et les itérations d'un CRF entièrement connecté (src: article de DeepLabV1)

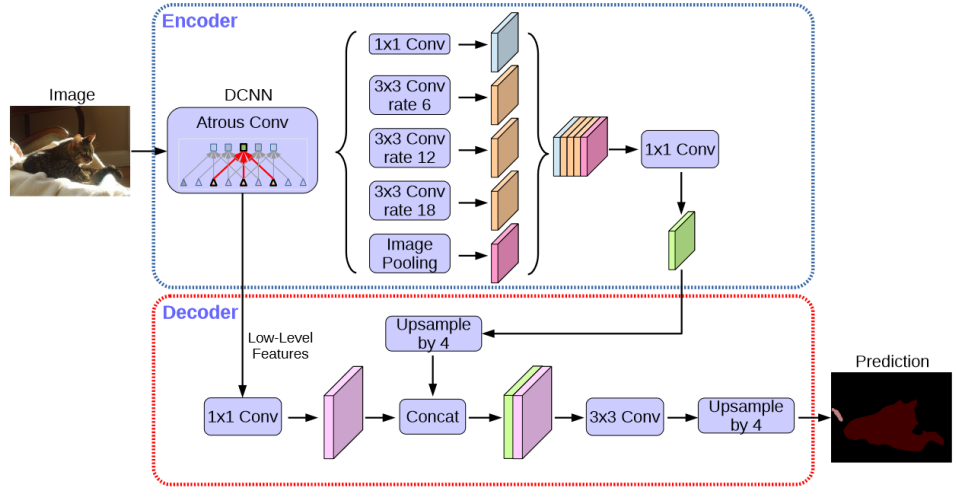
DeepLab V 2/3

- Amélioration de la V1:
    - On remplace l'intégration des différentes échelles par une approche pyramidale: **Atrous Pyramidal Pooling**
- Advantages:
    - Coût en calcul réduit car il n'y a plus qu'une seule branche
- DeepLabV3: Réseau de base plus profond



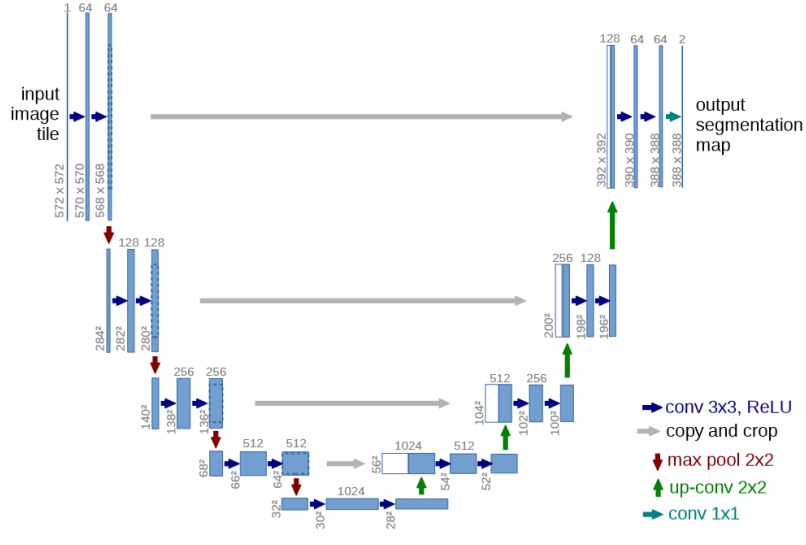
DeepLab V3+

- Fusion entre FCN et DeepLabV2/3 → architecture Encodeur-Decodeur + **atrous pyramid pooling**
    - Même baseline que DeepLabV3
    - Connections entre les échelles



U-Net

- Architecture encodeur-decodeur plus symétrique que DeepLabV3+
- augmentation du nombre d'échelle et de connections entre les échelles
- Convolution classique
- Sur-echantillonnage avec interpolation bilinéaire



Fonctions de coût

III.1 Cross-entropie

III.2 Focal Loss

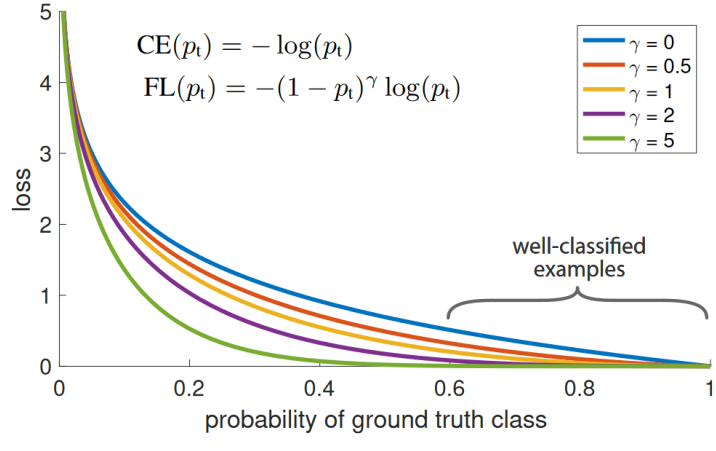
Cross-entropie

- Sortie du réseau → $H \times W \times K$
- On calcule une cross-entropie à chaque pixel

$$\mathcal{L}(I;\theta) = -\frac{1}{HW} \sum_{i,j} \delta\left(y_{i,j,k}, k\right) \log\left(p_{i,j,k}\right) \tag{1}$$

Focal loss

- Initialement développée pour de la détection d'objet, mais très efficace pour la segmentation
- Permet de régler les problèmes de déséquilibre entre les classes → évite le sur-apprentissage sur les classes sur-représentées
- **Idea**: Poids réduit pour les prédictions élevées



Métriques

IV.1 Précision par pixel

IV.2 Intersection sur l'union

IV.3 DICE

Précision par pixel

- Calcule le nombre de pixels correctement classifié par rapport au nombre total de pixels

$$PA = \frac{\sum_{j=1}^{K} n_{j,j}}{N} \tag{2}$$

- where:
    - $n_{i,j}$: Nombre de pixels avec comme véritable classe i et prédit comme appartenant à la classe j
    - N: Nombre total de pixels

Précision moyenne par pixel

- Problèmes de la précision par pixel : très biaisé par en cas de déséquilibre entre les classes
- **Solution**: moyenne de la précision par pixels sur toutes les classes

$$mPA = \frac{1}{K} \sum_{j=1}^{K} \frac{n_{j,j}}{t_j} \tag{3}$$

- where:
    - $n_{i,j}$: Nombre de pixels avec comme véritable classe i et prédit comme appartenant à la classe j
    - t_j: Nombre total de pixels avec comme véritable classe j

Intersection sur l'union

- Intersection sur l'union (**Intersection over Union - IoU**) or **Jaccard Index**
- Entre deux ensembles A et B, IoU est calculé comme:

$$IoU = \frac{|A \cap B|}{|A \cup B|} \tag{4}$$

- En utilisant les notations précédentes:

$$IoU = \frac{\sum_{\substack{i,j \\ i \neq j}} n_{j,j}}{\sum_{\substack{i,j \\ i \neq j}} \left(n_{j,j} + n_{i,j} + n_{j,i} \right)} \tag{5}$$

Mean Intersection over Union

- De même, on peut modifier l'IoU pour la moyenner sur les classes pour éviter les problèmes de déséquilibre

$$IoU = \frac{1}{K} \sum_{j=1}^{K} \frac{\sum_{\substack{i \\ i \neq j}} n_{j,j}}{\sum_{\substack{i \\ i \neq j}} \left(n_{j,j} + n_{i,j} + n_{j,i} \right)} \tag{6}$$

DICE

- En essence, similaire à l'IoU
- DICE est une mesure de distance entre 2 ensembles contrairement à l'IoU

$$IoU = \frac{2 \cdot |A \cap B|}{|A| + |B|} \tag{7}$$

- Using previous notations:

$$IoU = \frac{2 \cdot \sum_{\substack{i,j \\ i \neq j}} n_{j,j}}{\sum_{\substack{i,j \\ i \neq j}} \left(2 \cdot n_{j,j} + n_{i,j} + n_{j,i}\right)} \tag{8}$$

Post-traitement et tricks

V.1 Conditional Random Fields

Conditional Random Fields

- On modélise une image comme un champ aléatoire → chaque pixel appartient à un champ (l'image) et suit une distribution dépendant de cette image
- **Objectif**: Minimiser une function d'énergie θ (similaire à une fonction de coût) sur la probabilité d'appartenance des pixels
- Energie: une fonction évaluée à chaque point (ici les pixels) avec un ensemble de propriétés (continue, peu de variations, ...)

$$E(x) = \sum_i \theta_i(x_i) + \sum_{i \neq j} \theta_{i,j}(x_i, x_j) \tag{9}$$

- Monome → Cross-entropie
- Binome → Noyau Gaussian sur les pixels de l'image + de leur position
- Hyper-paramètres: écart-type de la Gaussienne pour contraindre imposer une contrainte localte (position des pixels et voisinages) or l'apparence (couleur des pixels)
- **Optimisation**: Descente de gradient

TP1 - Segmentation sémantique sur VOC2007

- Se rendre sur la page du cours
- Télécharger l'archive tp_segmentation.zip
- Compléter les **TODO**